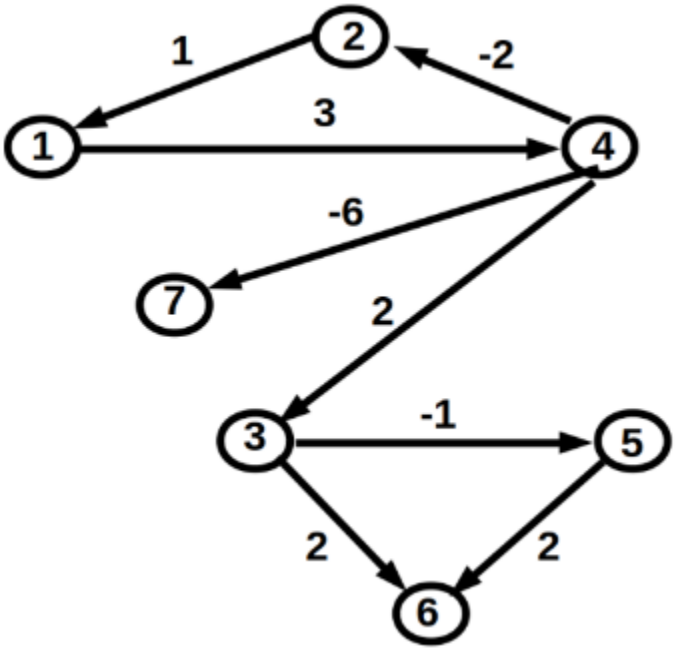
2020

Subiectul 1



1. Considerați graful neorientat și realizați o parcurgere DFS pornind din nodul 6 în urma căreia să se evidențieze timpii de start și finish pentru fiecare nod.
2. Considerați graful orientat și realizați o parcurgere BFS pornind din nodul 4 în urma căreia să evidențiați ordinea parcurgerii nodurilor.
3. Considerând graful neorientat și având costurile din figură, găsiți un arbore parțial de cost minim, folosind algoritmul lui Prim, considerând rădăcina arborelui nodul 2.
4. Considerând graful neorientat precizați care sunt componentele biconexe.
5. Pentru acest exercițiu ștergeți nodul 7 (împreună cu muchiile incidente). Considerând graful orientat și cu costurile din figură, să se aplice algoritmul A*, considerând starea inițială nodul 2 și starea finală nodul 6. Să se evidențieze, la fiecare pas, mulțimile OPEN și CLOSED, precum și modificările efectuate asupra tablourilor f și g.

   Euristica h folosită este dată în următorul tabel:
   node 1 234 56
   h 561 3 20

Observații despre redactare

- Pentru subpunctele 1 și 2 rezolvarea se poate face folosind un desen.
- Pentru subpunctele 3 și 5 arătați la fiecare pas care este conținutul structurilor de date folosite.
- Pentru subpunctul 4 nu trebuie justificare.

Subiectul 2 - ?

Subiectul 3

1. Se dă un graf neorientat conex reprezentând rețeaua de socializare Facebook (muchie x-y înseamnă x este prieten cu y). Gigel a primit astăzi o propunere de prietenie. El se gândește că ar putea să fie prietenul unui prieten sau prietenul prietenului etc. El se întreabă care este cel mai scurt lanț de la el la noul prieten. Descrieți un algoritm eficient pentru această problemă.

   Răspuns moodle:
   BFS (Gigel) si ne oprim cand ajungem in noul prieten, calculând distanțele pe parcurs. d[noul_prieten] este raspunsul.

2. Cum puteți afișa toate drumurile distincte între 2 noduri dintr-un graf neorientat aciclic?

   Răspuns moodle:
   Există un singur drum, deci o parcurgere e suficientă.

3. Se dă un graf orientat cu x noduri, având capacități numere din șirul Fibonacci. Se știe că există A surse și B noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

   Răspuns moodle:
   Se reunesc toate sursele într-o sursă fictivă S (muchie de la S la S de capacitate infinită) și toate destinațiile într-o destinație fictivă T (muchie de la T la T de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

4. Cum se poate calcula eficient al K-lea cel mai mic număr dintr-o secvență de N numere?

   Răspuns moodle:
   1p - n (folosind funcția de partiționare de la qsort)
   0.5p - n log k

5. Cum puteți să calculați numărul total de drumuri distincte între 2 noduri dintr-un graf orientat aciclic?

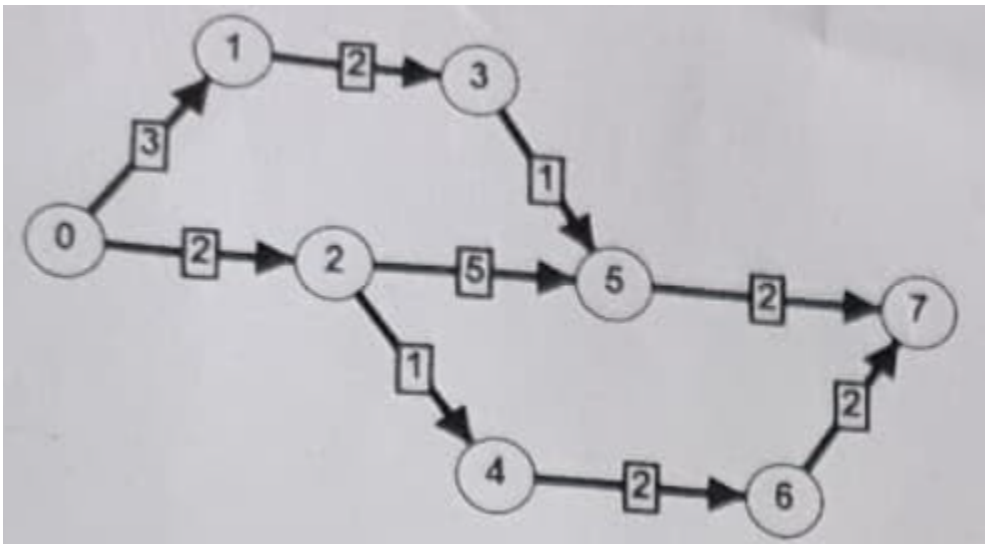
   Răspuns moodle:
   Sortare topologică apoi dinamică pentru a calcula numărul de drumuri pana la un anumit nod.

6. Se dă un graf neorientat cu N fortărețe și M drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze un singur drum a.î. Să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea acelui drum.

   Răspuns moodle:
   Găsim toate punțile. Gigel va alege puntea pentru care |număr noduri la stânga - număr noduri la dreapta| este minim.

2019 CB

Subiectul 1



1. Ignorând costurile de pe arce, să se calculeze folosind o parcurgere **DFS plecând din nodul 0** timpii de **descoperire** și de **finalizare** pentru fiecare nod. Puteți alege orice ordine de explorare a nodurilor.

2. Ignorând costurile de pe arce realizați o **sortare topologică** folosind algoritmul lui **Kahn**.

3. Considerând că ponderile de pe arce reprezintă distanțe între noduri, calculați drumul minim de la nodul **sursă 0** la nodul **destinație 7** folosind algoritmul **Bellman-Ford**. La fiecare pas, scoateți în evidență doar valorile care se schimbă față de pasul anterior.

4. Considerând ponderile de pe arce ca fiind **capacități**, determinați **fluxul maxim** în rețea folosind algoritmul **Edmonds-Karp**, știind sursa **0** și drena **7**. La fiecare pas, specificați **drumul de ameliorare** ales de algoritm și **cantitatea maximă de flux** ce poate fi transportată pe acest drum.

5. Determinați **numărul minim de înmulțiri de scalari** necesare pentru înmulțirea următoarelor matrici, precum și **parantezarea** necesară pentru a ajunge la acest număr optim: A1(4 linii, 10 coloane), A2(10, 3), A3(3, 12), A4(12, 20), A5(20, 7).

Subiectul 2

1. Care sunt proprietățile pe care trebuie să le satisfacă o problemă pentru a fi rezolvată prin tehnica programării lacome? Exemplificați cum se aplică acestea pentru problema construcției arborilor Huffman.

2. Care este semnificația vârfurilor d[v] pentru algoritmul lui Prim? Justificați pe scurt de ce muchiile alese de către algoritm sunt muchii sigure pentru arborele parțial minim construit până la acel moment.

3. Asemănări și deosebiri între algoritmii AC1 și AC3. Printre acestea, atingeți și aspecte legate de utilitate, funcționare, complexitate, de ce unul dintre cei 2 algoritmi este mai puțin eficient și cum se poate rezolva această problemă.

4. Care este complexitatea algoritmului Edmonds-Karp? Justificați pe scurt cum se poate demonstra această limită de complexitate.

5. Definiți ce înseamnă admisibilitatea și dominanța pentru algoritmii de tip A*. Pentru jocul 8-puzzle discutat la curs, dați exemplu 3 euristici nebanale (care să nu fie constante, ci să depindă de pozițiile pieselor din starea curentă și din starea finală) dintre care una să nu fie admisibilă. Justificați pe scurt de ce euristicile alese sunt admisibile (sau nu), precum și care dintre cele două euristici admisibile o domină pe cealaltă.

Subiectul 3

După cum se știe foarte bine, Harap-Alb are multe calități, printre care cunoașterea perfectă a hărții împărăției, precum și stăpânirea elementelor de bază despre algoritmi. Împărăția poate fi modelată ca un graf orientat cu **n** noduri - orașele și **m** arce - drumurile, și se dau și toate

costurile drumurilor. Harap-Alb merge întotdeauna de la reședința Împăratului Roșu, nodul s, către cea a Împăratului Verde, nodul f, folosind doar drumuri de cost minim (este un tip practic și folosește cunoștințele de algoritmi învățate la PA). Stabiliți toate muchiile pe care poate sta Spânul, în așteptarea lui Harap-Alb, pentru a fi sigur că nu ratează întâlnirea cu acesta. Atenție: între cele 2 reședințe pot fi mai multe drumuri care să aibă același cost minim.

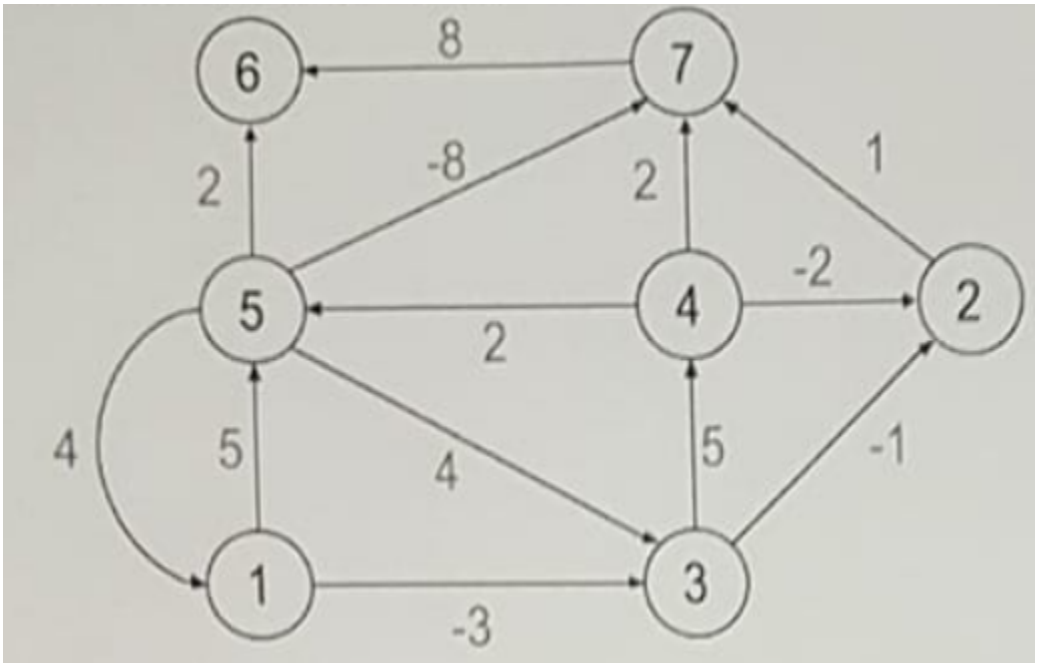
Exemplu: pe tablă

Se cere **o soluție optimă.**

a. Explicați **ideea** de rezolvare. Spuneți ce **structuri de date / algoritmi** clasici veți folosi (dacă folosiți).
b. Schițați **pseudocodul**, fără a intra în detalii inutile. Subpunctele a și b pot fi rezolvate împreună.
c. Determinați **complexitatea temporală** și **spațială** pentru soluția găsită. Justificați pe scurt.

2017 CA

Subiectul 1



Se dă graful din imaginea alăturată.

1. Să se aplice o parcurgere în adâncime din nodul 1 alegând nodurile în ordinea crescătoare a id-urilor, acolo unde este cazul. Evidențiați pentru fiecare nod timpul de descoperire și de finalizare.

2. Pornind de la parcurgerea de la punctul 1, determinați componentele tare conexe ale grafului, parcurgând graful transpus și calculând timpii de descoperire și finalizare.

3. Aplicați algoritmul Bellman-Ford, considerând că 1 reprezintă nodul sursă. Să scrie valorile lui d după fiecare din cei |V|-1 pași.

4. Considerând graful neorientat, aplicați algoritmul Kruskal. Specificați muchiile selectate la fiecare pas respectiv arborele final determinat de algoritm.

5. Să se aplice algoritmul A*, considerând starea inițială nodul s = 1 și starea finală nodul t = 6. Euristica folosită este h(v) = {min(abs(cost(v, v') | (v, v') este muchie în graf} pentru v != t si h(t) = 0. Să se calculeze valorile euristicii h. Să se evidențieze, la fiecare pas, mulțimile OPEN și CLOSED, precum și modificările efectuate asupra f și g.

Subiectul 2

1. Să se spună ce algoritm făcut la curs poate să determine eficient care este **drumul de lungime maximă într-un graf aciclic orientat** cu lungimi pe arce și să se schițeze ideea folosirii acestuia și modificările necesare pentru rezolvare.

2. Să se spună ce schemă de algoritm făcută la curs poate să determine, cât mai eficient posibil, dacă un careu de rebus are o asignare posibilă de cuvinte (nu se consideră definițiile cuvintelor). Să se schițeze ideea rezolvării. Se presupune că există un dicționar în care cuvintele sunt reprezentate prin vectori de caractere. (un careu de rebus este o matrice **m x n** cu unele celule goale. Toate celelalte celule trebuie să primească câte o literă astfel încât secvențele maximale de celule pe orizontală și verticală să fie cuvinte valide).

3. Comparați A* și parcurgerea în adâncime.

4. Explicați un exemplu simplu de ce algoritmul Dijkstra merge doar pentru costuri pozitive pe arce.

5. Fie un graf neorientat cu **n** noduri și **a** arce. Care este numărul minim și maxim de punți și puncte de articulație ale acestui graf? Justificați. Dar dacă graful are puncte de articulație dar nu are punți?

Subiectul 3

Se dau numerele naturale n, s și b1, b2, .., bn. Să se determine câte ecuații de forma a1 + a2 + … + an = s cu ai natural și ai >= bi (pentru orice i = 1…n) există?

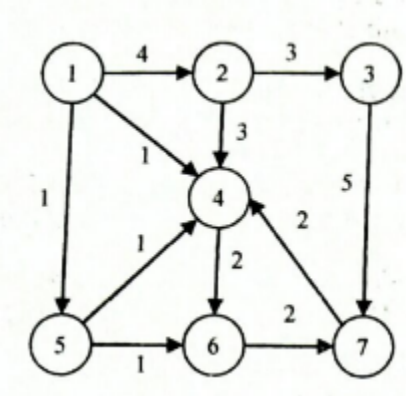
a. Explicați mai întâi care este ideea voastră de rezolvare (în cuvinte, încercați să oferiți cât mai multe detalii).

b. Spuneți ce structuri de date, tehnici de programare și/sau algoritmi clasici (prezentați la curs) veți folosi.

c. Schițați pseudocodul, fără a intra în detalii (inutile). Atenție: se dorește o complexitate cât mai bună. Determinați complexitatea soluției propuse.

2017 CB

Subiectul 1



1. Să se aplice o parcurgere în adâncime pornind din nodul 1, specificând timpii de descoperire și de finalizare pentru fiecare nod.

2. Evidențiați componentele tare conexe ale grafului aplicând algoritmul lui Kosaraju (puteți pleca de la parcurgerea în adâncime de la punctul 1)

3. Considerând graful **neorientat**, să se aplice algoritmul lui Kruskal, scoțând în evidență muchiile alese de algoritm și ordinea alegerii lor.

4. Aplicați algoritmul lui Dijkstra considerând nodul sursă s = 1, iar la fiecare pas scrieți valorile din vectorul de distanțe d și scoateți în evidență nodul extras de algoritm din coada de priorități.

5. Determinați fluxul maxim în graf folosind algoritmul Edmonds-Karp, știind că sursa este nodul 1, iar drena este nodul 7. La fiecare pas, specificați vârfurile care fac parte din drumul de ameliorare ales de algoritm, precum și cantitatea de flux ce poate fi transportată pe acest drum.

Subiectul 2

1. Dați exemplu de o problemă care se poate rezolva, eventual cu constrângeri diferite asupra datelor de intrare, atât folosind programare dinamică, cât și cu programare

lacomă. Explicați diferențele și asemănările între cei 2 algoritmi (inclusiv complexitatea lor).

2. Aplicați minimax și evidențiați tăieturile alfa-beta pentru arborele de joc alăturat. Influențează ordinea de evaluare a nodurilor numărul de tăieturi dintr-un arbore de joc?

3. Care este numărul minim, respectiv maxim, de componente tare conexe ale unui graf care permite o sortare topologică? Justificați

4. Fie un algoritm A* ghidat de următoarea euristică, h(n) = 0 pentru orice n din V, pentru problema 8-puzzle discutată la curs. Explicați care este numărul maxim de noduri care sunt scoase din mulțimea CLOSED pentru a fi reintroduse în mulțimea OPEN în timpul găsirii unei secvențe de pași de la o configurație inițială a problemei la o configurație finală. Explicați pe scurt principiul de funcționare al algoritmului pompare preflux (push-relabel). Care este înălțimea maximă la care poate ajunge un nod în timpul rulării algoritmului? De ce este utilă/necesară această limită?

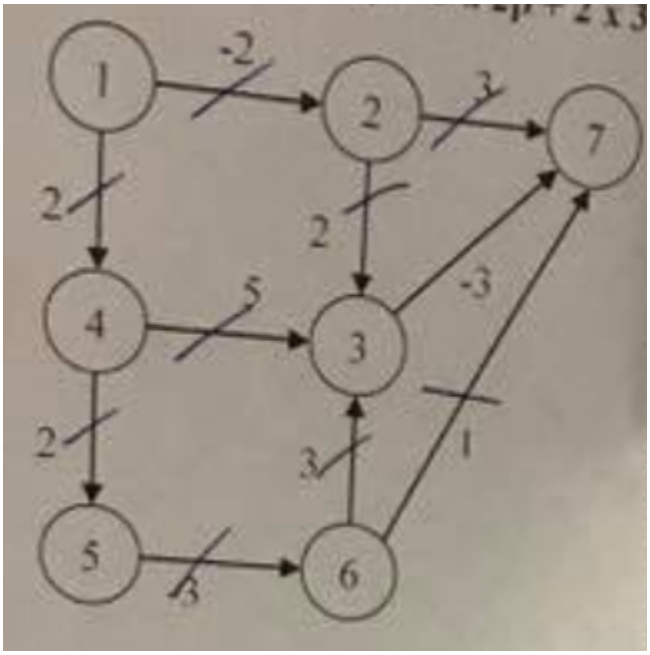
Subiectul 3

Gigel are o grădină cu meri care este modelată ca o matrice de dimensiune n x n, iar în fiecare celulă este o cantitate de mere cunoscută, **livada[i][j]**. Știind că Gigel intră în livadă prin colțul stânga-sus și merge la fiecare pas, ori în jos ori la dreapta cu o celulă, ieșind prin colțul din dreapta-jos, ajutați-l să găsească calea pe care poate culege cele mai multe mere. Afișați acest număr maxim de mere culese și calea aleasă. (Hint: **n <= 1000, -10^4 <= livada[i][j] <=10^4**).

a. Explicați mai întâi care este ideea voastră de rezolvare (în cuvinte, încercați să oferiți cât mai multe detalii).
b. Spuneți ce structuri de date, tehnici de programare și/sau algoritmi clasici (prezentați la curs) veți folosi.
c. Schițați pseudocodul, fără a intra în detalii (inutile). Atenție: se dorește o complexitate cât mai bună. Determinați complexitatea soluției propuse.

2017 CC

Subiectul 1



Se dă graful din imaginea alăturată.

1. Considerând graful **orientat** și **fără costuri**, să se aplice o parcurgere în lățime pornind din nodul 1.

2. Considerând graful **orientat** și **fără costuri**, să se determine o sortare topologică a grafului.

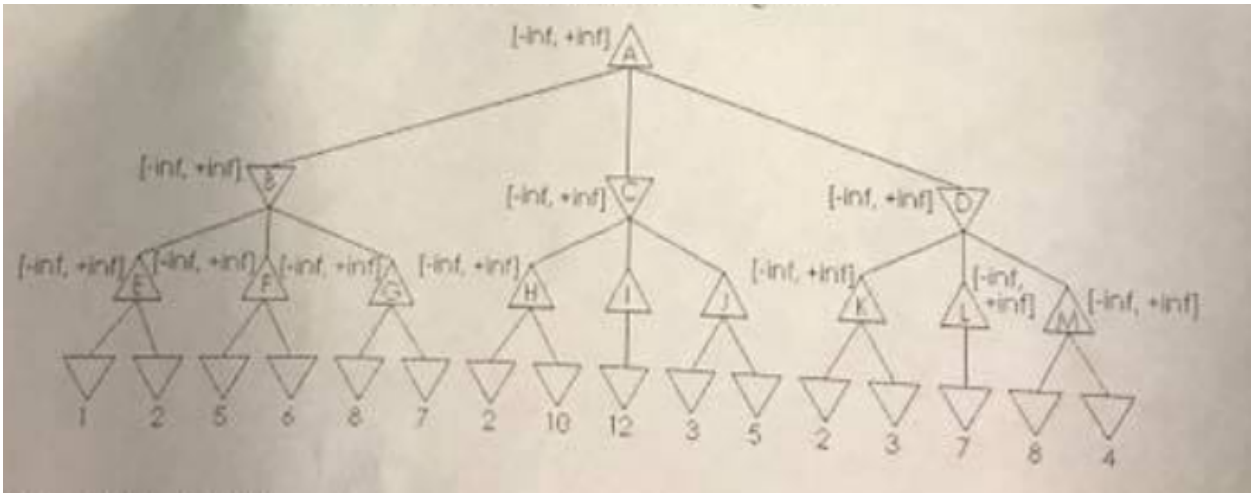
3. Considerând graful **neorientat** și **pozitivând costul muchiilor negative**, prin considerarea modulului acolo unde este cazul, aplicați algoritmul lui Dijkstra pornind din nodul 1 (la fiecare pas, evidențiați valoarea fiecărui nod aflat în coada de priorități, precum și nodul extras din coadă).

4. **Pozitivând costul muchiilor negative**, prin considerarea modulului acolo unde este cazul, să se aplice algoritmul A*, considerând starea inițială nodul s = 1 și stările finale nodurile t1 = 6 și t2 = 7. Euristica folosită este h(v) = {min(abs(cost(v, v') | (v, v') este muchie în graf} pentru v **not in** {t1, t2} și h(t1) = h(t2) = 0. Evidențiați la fiecare pas mulțimile OPEN și CLOSED și modificările efectuate asupra lui f și g.

5. Determinați fluxul maxim în graf folosind algoritmul Edmonds-Karp, știind că sursa este nodul 1, iar drena este nodul 7. La fiecare pas, specificați vârfurile ce fac parte din drumul de ameliorare ales de algoritm, precum și cantitatea de flux ce poate fi transportată pe acest drum. Capacitățile maxime ale arcelor sunt valorile din graf **pozitivate** la fel ca mai sus (prin considerarea modulului acolo unde este cazul).

Subiectul 2

1. Care este numărul maxim de pompări săturate în cadrul algoritmilor de tip preflux (push-relabel) folosiți pentru determinarea fluxului maxim. Argumentați pe scurt
2. Cum se definește complexitatea unui algoritm de tip Monte Carlo?
3. Demonstrați corectitudinea algoritmului Kruskal.
4. Comparați algoritmii de tip greedy cu cei de programare dinamică. Dați 3 exemple din fiecare categorie.
5. Aplicați algoritmul alfa-beta pe arborele de mai jos, analizând nodurile de la stânga la dreapta. Se cer: identificarea valorii nodului A și evidențierea nodurilor tăiate de către algoritm.



Subiectul 3

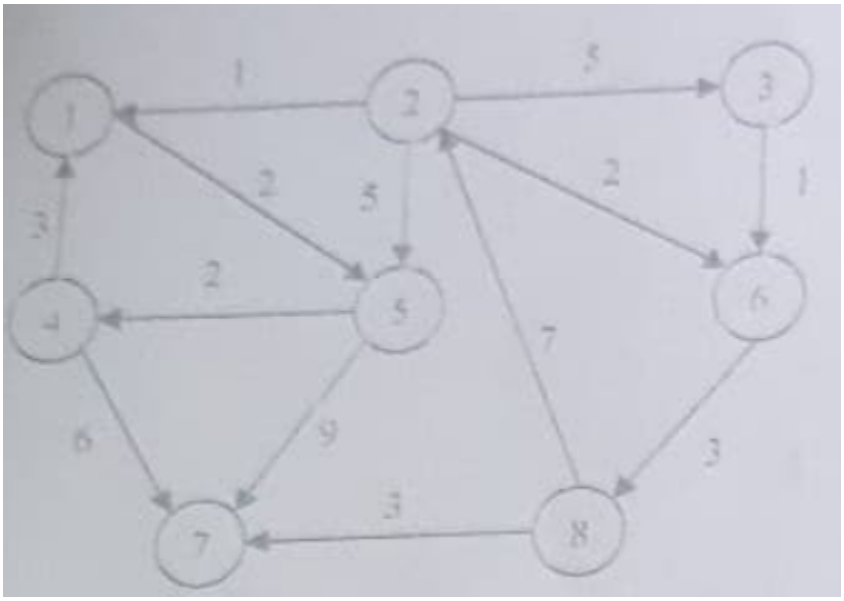
În New Bucharest, există un număr de străzi pe care se poate circula în ambele direcții și care au drept scop asigurarea transportului rapid dinspre periferie spre punctele centrale ale orașului. Vi se dau cele n intersecții din oraș și străzile care asigură o legătură directă între 2 intersecții. În plus, se știe că pentru a evita costurile suplimentare rețeaua de străzi nu conține cicluri. Scopul vostru este să determinați care este numărul maxim de străzi prin care trebuie să treceți pentru a traversa orașul, precum și să găsiți cele 2 intersecții care determină calea cea mai lungă. Observație: ca în orice oraș, unele străzi principale sunt fundături, așa că pot exista intersecții din care să plece o singură stradă.

a. Explicați mai întâi care este ideea voastră de rezolvare (în cuvinte, încercați să oferiți cât mai multe detalii).
b. Spuneți ce structuri de date, tehnici de programare și/sau algoritmi clasici (prezentați la curs) veți folosi.

c. Schițați pseudocodul, fără a intra în detalii (inutile). Atenție: se dorește o complexitate cât mai bună. Determinați complexitatea soluției propuse.

Ceva restanță

Subiectul 1



Se dă graful din imaginea alăturată.

1. Să se aplice o parcurgere în adâncime specificând timpii de descoperire și de finalizare pentru fiecare vârf din graf.

2. Aplicați primii 2 pași ai algoritmului Floyd-Warshall, specificând matricile D(0), D(1), D(2). Dacă vă este mai ușor, la fiecare pas puteți specifica doar acele elemente care se modifică față de pasul anterior.

3. Ignorând muchia 8-2, afișați drumurile de ameliorare folosite de către algoritmul Edmonds-Karp pentru sursa s = 1 și drena t = 7. La final spuneți care este fluxul maxim transportat în cadrul rețelei.

4. Considerând graful neorientat, aplicați algoritmul lui Prim pornind din nodul 1. La fiecare pas, scrieți valorile pentru df[u] pentru fiecare nod și menționați explicit ce muchie este folosită în cadrul AMA.

5. Scrieți algoritmul A* considerând sursa s = 2 și destinația d = 7, iar euristica este definită ca h(v) = max{w(v, u) | u in V} și h(d) = 0. Scrieți valoarea funcției euristică pentru fiecare nod și apoi specificați secvența de pași până se ajunge la destinație. La fiecare pas, pentru fiecare nod din OPEN și CLOSED menționați valorile f(u) și g(u), precum și nodul extras din OPEN la pasul respectiv.

Subiectul 2

1. Cum se poate modifica algoritmul lui Dijkstra dacă graful este aciclic orientat? Care este complexitatea algoritmului modificat propus?

2. Care sunt principalele asemănări și diferențe între algoritmii Kruskal și Prim?

3. Un robot de căutare montană are un sistem de vedere artificială care poate detecta vârfurile munților din ceea ce detectează camera sa de luat vederi. Un algoritm pentru dirijarea acestui robot generează la fiecare iterație o deplasare a robotului cu o distanță constantă către cel mai înalt vârf detectat. Descrieți cel puțin 2 situații în care algoritmul este ineficient sau chiar nu găsește soluția optimă.

4. La parcurgerea în adâncime a unui graf neorientat cu algoritmul de la curs, în ciclul cel mai superficial, care apelează procedura recursivă, se fac 2 iterații. Ce se poate spune despre graf?

5. Ce înseamnă că o euristică pentru algoritmul A* este admisibilă? Ce s-ar întâmpla, dacă această condiție nu s-ar respecta?

Subiectul 3

Primarul orașului vostru natal are nevoie de ajutor pentru a rezolva următoarea problemă. Harta orașului este modelată ca un graf orientat cu n vârfuri reprezentând intersecțiile principale ale orașului și m arce reprezentând drumurile între aceste intersecții. Mai mult, pentru fiecare dintre drumuri este cunoscut d(u, v) = lungimea drumului între intersecțiile u și v. Cum primarul are cunoștințe de bază în informatică, el știe să determine drumul cel mai scurt între două intersecții oarecare din oraș - sursa s și destinația t - folosind un algoritm clasic. Însă voi trebuie să îl ajutați să descopere al doilea cel mai scurt drum între aceleași două intersecții, s și t, folosind un algoritm cât mai eficient, în acest context, definim al doilea cel scurt drum ca fiind drumul de la s la t de cost minim ce are cel puțin o muchie diferită față de cel mai scurt drum între aceste 2 vârfuri.

a. Explicați mai întâi care este ideea voastră de rezolvare (în cuvinte, încercați să oferiți cât mai multe detalii).
b. Spuneți ce structuri de date, tehnici de programare și/sau algoritmi clasici (prezentați la curs) veți folosi.
c. Schițați pseudocodul, fără a intra în detalii (inutile). Atenție: se dorește o complexitate cât mai bună. Determinați complexitatea soluției propuse.